

Mi primera app Android

Realizado por Javier Gallego
Narváez



Índice

Parte A: Análisis del desarrollo móvil	3
Ventajas e inconvenientes	3
Ventajas	3
Desventajas	3
Limitaciones técnicas	3
Elección tecnológica	4
Ciclo de vida de una activity	5
Parte B: Especificación de tu app	6
Idea de la aplicación y público objetivo	6
Requisitos funcionales	6
Requisitos no funcionales	7
Casos de uso	7
Caso de Uso: Consultar ficha de detalle	7
Caso de Uso: Marcar producción como vista o pendiente	8
Parte C: Entorno de desarrollo	9
Estructura del proyecto	10
Estructura y directorios clave	10
Conexión entre Activity (Java) y Layout (XML)	10



Parte A: Análisis del desarrollo móvil

Ventajas e inconvenientes

Ventajas

- Portabilidad: Como el usuario siempre lleva el móvil consigo, podrá usar la app en cualquier momento y lugar
- Acceso directo al hardware y sensores: Cierta facilidad a la hora de implementar funciones nativas del dispositivo
- Notificaciones: Posibilidad de comunicación con el usuario al instante mediante notificaciones

Desventajas

- Dependencia de las tiendas de aplicaciones: Si hay algún error, hay que realizar cada actualización empaquetandola, firmandola, enviandosela a Google y esperando a que pase el proceso de revisión y aceptación
- Coste y esfuerzo de desarrollo por plataformas: Si quiero que mi app esté disponible para todo el mundo, hay que programarla dos veces, para que sea accesible tanto en Android como en iPhone
- Ciclo de vida: Si el usuario abre otra aplicación pesada mientras se está usando nuestra aplicación, el sistema operativo Android puede matar la aplicación en segundo plano para liberar memoria RAM, obligando al programador a guardar el estado continuamente

Limitaciones técnicas

- Batería: Al tener autonomía limitada, la aplicación evita tareas en segundo plano y solo consulta datos cuando el usuario lo pide para no gastar batería



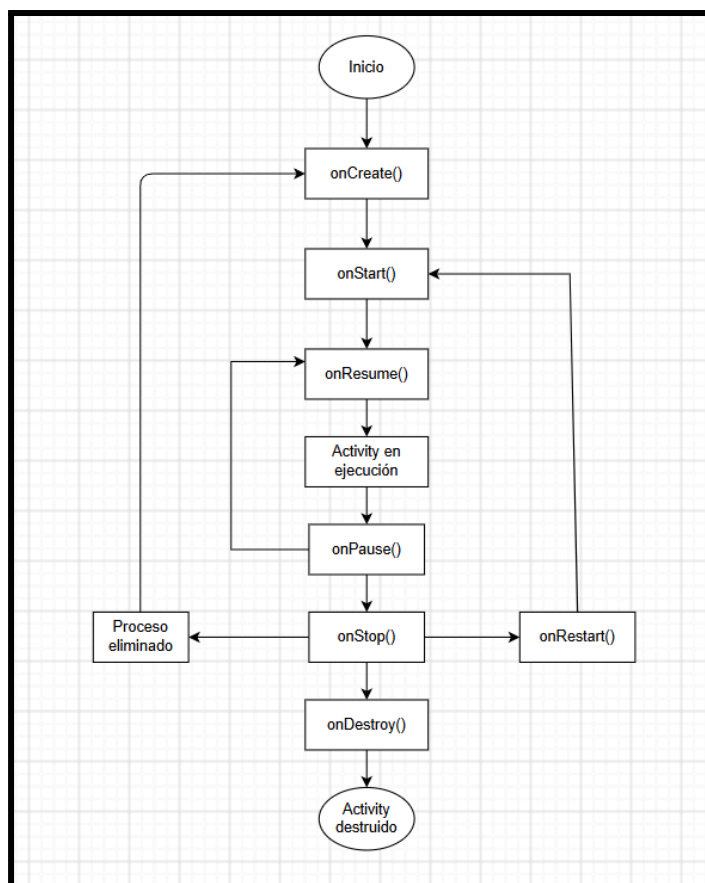
- Memoria RAM: Como la memoria es reducida, se optimizan y comprimen las carátulas para evitar cierres inesperados por falta de memoria
- Conectividad: Ante cortes de cobertura, la aplicación guarda las películas las películas vistas en local para funcionar sin Internet
- Fragmentación de pantallas: Debido a la variedad de dispositivos, se usan unidades adaptables para que el contenido para que el contenido nunca se corte

Elección tecnológica

Enfoque	Tecnologías	Ventajas	Inconvenientes	Rendimiento
Nativo	Java/Kotlin (Android), Swift(iOs)	Máximo rendimiento y acceso total a lo más reciente del sistema	Mayor coste y tiempo	Excelente (Nativo directo)
Híbrido	Ionic, Cordova, Capacitor	Un único código para todas las plataformas	Rendimiento inferior, peor experiencia visual táctil y dependencia de navegador web interno	Bajo / Medio
Multiplataforma	Flutter (Dart), React Native (JS)	Un solo código base casi nativo y mejor rendimiento que las híbridas	Mayor peso de la aplicación, puentes de comunicación y retraso en adoptar novedades	Alto

Mi app se desarrollará en nativo Android porque necesita respuestas inmediatas sin sobrecarga, gestión al detalle de la memoria RAM al cargar imágenes y control del ciclo de vida de las *activities*.

Ciclo de vida de una activity



Si, al usar mi app, el usuario recibe una llamada, la app pasa a segundo plano, pausando tareas no esenciales.

Si se gira el móvil mientras se usa la app, se destruye la activity por completo y se vuelve a crear para adaptarse al ancho de la pantalla horizontal.

Si se pulsa el botón Atrás, la activity finaliza su ciclo, retirándose de la pila de pantallas y liberando de la memoria RAM todos los elementos visuales.



Parte B: Especificación de tu app

Idea de la aplicación y público objetivo

Marvel Tracker es una aplicación móvil nativa de Android diseñada para los fans de las películas y series de Marvel. Su propósito principal es servir como un organizador personal para seguir el orden de las películas y series de marvel, permitiendo al usuario registrar cuales ha visto, cuales tiene pendientes y ver información.

Sus objetivos principales son:

- Facilitar la consulta rápida del catálogo de producciones de Marvel
- Proporcionar información de los diferentes títulos
- Garantizar funcionamiento fluido y autónomo para consultar el progreso sin depender de cobertura de red

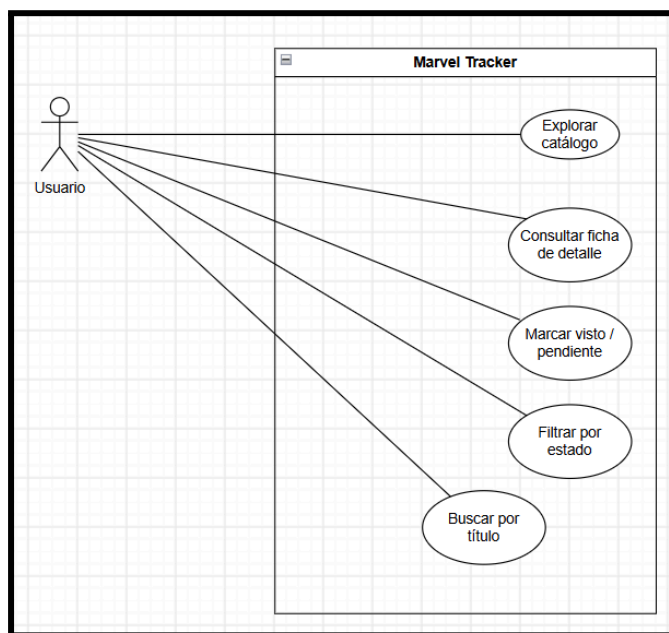
Requisitos funcionales

- RF-01: La aplicación debe mostrar una lista estructurada de todas las producciones (Prioridad alta)
- RF-02: Al pulsar en cada producción, se abrirá una vista hacia toda la información (Prioridad alta)
- RF-03: El usuario puede marcar cada producción como visto o pendiente (Prioridad alta)
- RF-04: La aplicación debe permitir al usuario filtrar las producciones (Prioridad baja)
- RF-05: La aplicación debe permitir buscar producciones por nombre

Requisitos no funcionales

- RNF-01: La aplicación debe desarrollarse en Android nativo empleando Java, garantizando compatibilidad con dispositivos que ejecuten Android 8.0
- RNF-02: El progreso de visualización debe de guardarse de forma local en el dispositivo, permitiendo usar la app sin cobertura
- RNF-03: La interfaz debe adaptarse a diferentes densidades y orientaciones de pantalla
- RNF-04: Las transiciones entre pantallas deben completarse en menos de un segundo

Casos de uso



Caso de Uso: Consultar ficha de detalle

- **Objetivo:** Ver la información completa de una película o serie seleccionada del catálogo.



- **Actor:** Usuario.
- **Requisito previo:** Estar en la pantalla del catálogo principal.
- **Flujo clave:**
 1. El usuario pulsa un título.
 2. La app abre la pantalla de detalle mostrando banner/carátula, título, año, fase y sinopsis.
 3. El usuario puede regresar al catálogo con el botón de retroceso sin perder el estado de la lista.

Caso de Uso: Marcar producción como vista o pendiente

- **Objetivo:** Cambiar y registrar el estado de visualización de una producción.
- **Actor:** Usuario.
- **Requisito previo:** Estar en la ficha de detalle o sobre un elemento interactivo del catálogo.
- **Flujo clave:**
 1. El usuario pulsa el botón/casilla de estado («Visto»).
 2. La interfaz cambia visualmente de inmediato.
 3. El nuevo estado se guarda en almacenamiento local (*SharedPreferences* o base de datos) para persistir sin conexión.
- **Alternativa:** Si ya estaba vista, al pulsar vuelve a «Pendiente» y se actualiza el guardado local.



Parte C: Entorno de desarrollo



Dispositivo	Tipo	Resolución	Densidad	Versión de API / Sistema
Google Pixel 8	Teléfono móvil	1080 X 2400 px	420 dpi	API 34



Estructura del proyecto

Estructura y directorios clave

- **AndroidManifest.xml**: Actúa como el descriptor global de la app ante el sistema operativo. En él se declaran los componentes principales (Activities), los permisos necesarios y cuál es la pantalla de inicio mediante su filtro de intención (LAUNCHER).
- **java/**: Directorio donde reside la lógica funcional de la aplicación organizada en paquetes. Aloja los controladores de pantalla (MainActivity), los modelos de datos y la gestión de eventos.
- **res/ (Recursos estáticos)**: Centraliza todos los elementos gráficos y de configuración visual fuera del código compilado:
 - layout/: Interfaces visuales estructuradas en ficheros XML.
 - values/: Valores reutilizables como cadenas (strings.xml), colores (colors.xml) y temas visuales (themes.xml).
 - drawable/ y mipmap/: Imágenes, iconos vectoriales e iconos de inicio del sistema.
- **build.gradle**: Scripts que automatizan la compilación, gestionan librerías externas y definen las versiones de compatibilidad del SDK (minSdk, targetSdk).

Conexión entre Activity (Java) y Layout (XML)

Android separa estrictamente el diseño de la lógica mediante un ciclo de tres pasos:

1. **Definición visual (XML)**: Se diseña la interfaz asignando identificadores únicos a cada elemento con `android:id="@+id/..."`.



2. **Carga en memoria (Inflado)**: Dentro del método onCreate() de la Activity, la llamada a setContentView(R.layout.activity_main) carga e interpreta el XML en pantalla apoyándose en la clase generada R.
3. **Interacción (Java)**: Se enlaza el código con los componentes visuales mediante findViewById(R.id...), permitiendo escuchar eventos de pulsación (setOnClickListener) y actualizar datos de forma dinámica.